Université Paris City

UFR Sciences Fondamentales et Biomédicales

M1 Cybersécurité et e-santé

UE : Réseaux (RESOIP)

Compte Rendu du TP4 Mail

Emailing Security

Réalisé par :

Rayane

RAAB

22418402

Année universitaire : *2025 / 2026*

I. Introduction :

Ce TP porte sur la mise en place d'un système de messagerie "entreprise", puis sur son analyse et sa sécurisation au travers de plusieurs scénarios d'accès (Webmail, clients mail) et de mécanismes de chiffrement.

L'idée est d'observer ce qui transite réellement sur le réseau (identifiants, contenu, ports/protocoles) avant et après sécurisation, afin de comprendre les risques et les protections.

Contexte et Objectifs : Le TP vise à installer, analyser et sécuriser un système de messagerie d'entreprise, Il est demandé de répondre aux questions et, si possible, d'illustrer avec des captures (Wireshark, TCPView, côté serveur et côté client).

Architecture générale attendue : Un serveur de messagerie est installé sur la machine principale (dans mon cas Windows) avec Kerio Mail Server, en définissant un domaine d'entreprise (dans mon cas raabsociety.fr) et un nom de serveur du type (mailhost.raabsociety.fr). Ensuite, plusieurs comptes utilisateurs (Alice, Bob et Marie) sont créés et serviront ensuite à tester l'envoi/réception selon différents modes d'accès.

Plan du TP : Le TP enchaîne des accès non chiffrés (webmail en HTTP, POP3/IMAP/SMTP classiques) pour vérifier si l'on peut intercepter identifiants et messages, puis des accès chiffrés (SMTPS, IMAPS, POP3S) pour constater l'amélioration. Enfin, il aborde la sécurité de bout en bout avec PGP (signature et chiffrement), qui protège le message indépendamment du canal de transport.

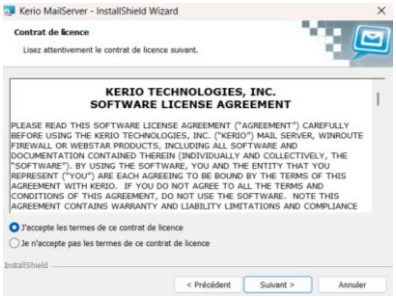
II. Partie 1 – Installation du système de messagerie :

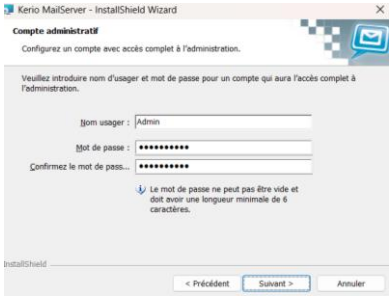
Dans cette partie, l'objectif est d'installer et configurer un serveur de messagerie d'entreprise sur le poste Windows (Hôte), pour ce faire, on utilise Kerio Mail Server, et puis on créera les comptes utilisateurs nécessaires.

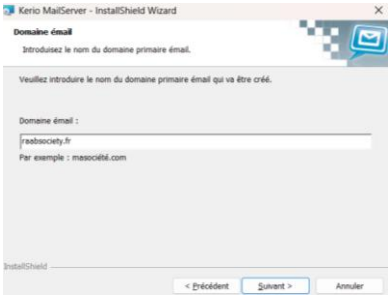
Un nom de domaine d'entreprise (**raabsociety.fr)** est défini et le serveur doit être identifié sous la forme **mailhost.raabsociety.fr**.

1.1. Installation et configuration de Kerio sur mon hôte Windows :

Le serveur Kerio Mail Server (fourni avec le TP) a été installé sur la machine hôte Windows tout en suivant l'installation par défaut proposée par l'installateur. Le domaine retenu pour l'entreprise est **raabsociety.fr**, et le serveur de messagerie a été nommé : **mailhost.raabsociety.fr**, conformément au format demandé dans l'énoncé… comme le montre les illustrations ci-dessous :







Le réseau de notre lab a été mis en place sous VirtualBox, en mode Host-Only, afin de permettre aux VM clientes Kali Linux d'accéder au serveur Windows via un réseau isolé, tel que :

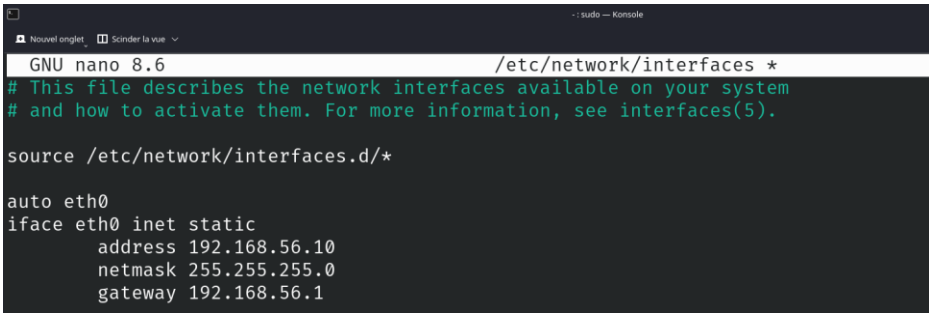
- Interface Host-Only Windows: **VirtualBox Host-Only Ethernet Adapter**, adresse IP: **192.168.56.1**, Masque: **255.255.255.0**, DHCP server: **disabled**

- VMKALI du client 1 : Adresse IP : **192.168.56.10/24**
- VMKALI du client 2 : Adresse IP : **192.168.56.11/24**

Pour avoir cela, on doit mettre les machines virtuelles de nos clients sur la même interface réseau que celle de Windows qui est celle de l'Host-Only, et puis configurer manuellement l'adresse IP de chaque client indépendamment (car le serveur DHCP est désactivé sur le Host-Only), pour ce faire on doit modifier le fichier **interfaces** se trouvant au niveau de **/etc/network/,** en utilisant la commande suivante :

sudo nano /etc/network/interfaces

Par exemple, sur notre VMKALI1 du client n°1 on aura la configuration réseau suivante au niveau de ce fichier :

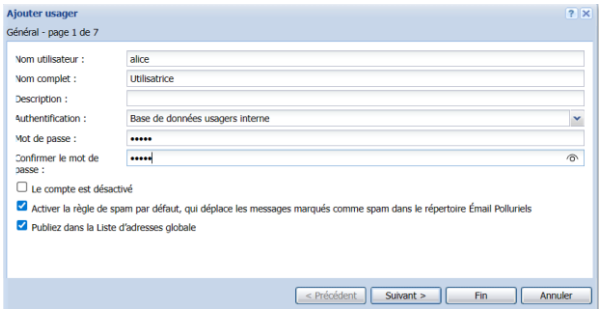


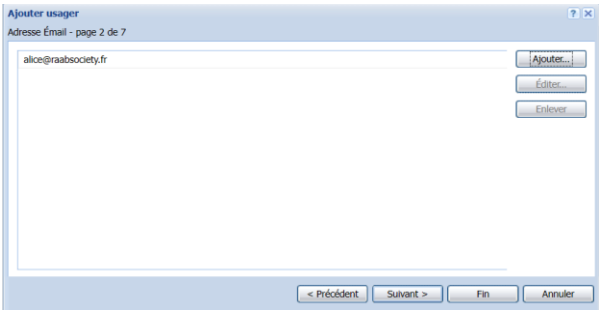
1.2. Création des comptes utilisateurs (Kerio) :

Trois comptes utilisateurs ont été créés sur le serveur, conformément au format imposé par le tp :

- Compte 1 : alice@raabsociety.fr / Mot de passe : user1
- Compte 2 : bob@raabsociety.fr / Mot de passe : user2
- Compte 3 : marie@raabsociety.fr / Mot de passe : user3

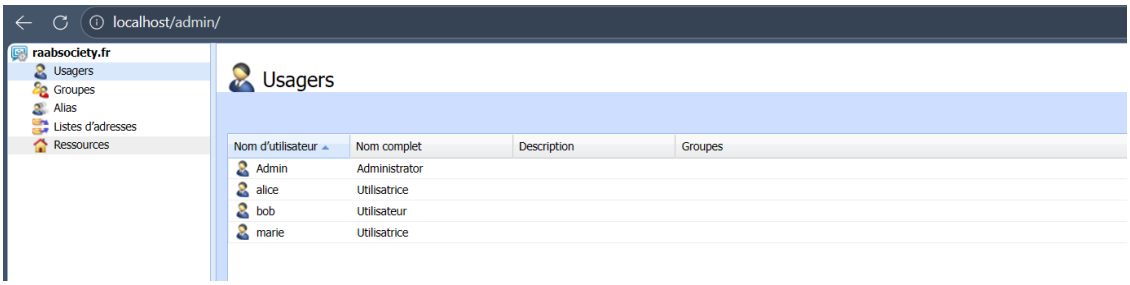
Pour créer ces comptes utilisateurs, on doit taper l'url suivante : **http://localhost/admin** , on peut utiliser l'identifiant admin et le même mot de passe choisi lors de l'installation de Kerio. Et puis, dans l'onglet Usagers, On clique sur ajouter, et on suit les illustrations suivantes :





On veille bien à laisser tout autre paramètre à sa configuration par défaut…

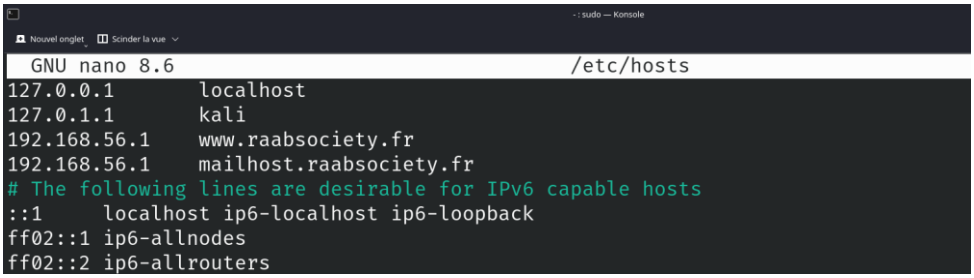
A la fin de cette, on aura obtenu quelque chose de ce type :



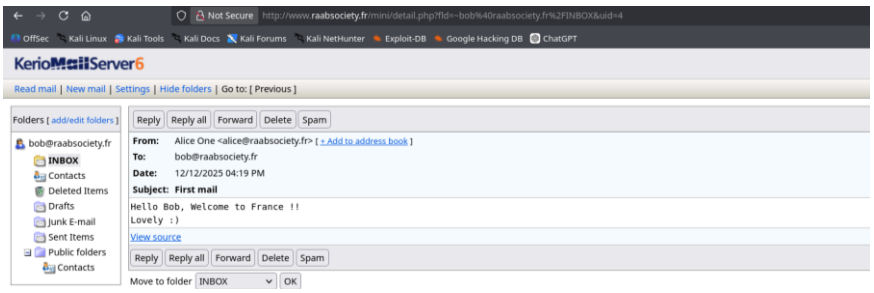
III. Partie 2 – WebMail :

- **Q1 : Configuration du fichier hosts + Accès Webmail en HTTP : 80**

Sur le poste client, le fichier **/etc/hosts** a été modifié afin d'associer l'alias **www.raabsociety.fr** à l'adresse IPV4 du serveur webmail 192.168.56.1, ainsi en tapant l'url http://www.raabsociety.fr sur le navigateur du client, ça nous dirigera sur la page de connexion à la messagerie, on va aussi associer **mailhost.raabsociety.fr** à la même adresse IPV4 suivante 192.168.56.1, pour ce faire on utilise la syntaxe suivante :



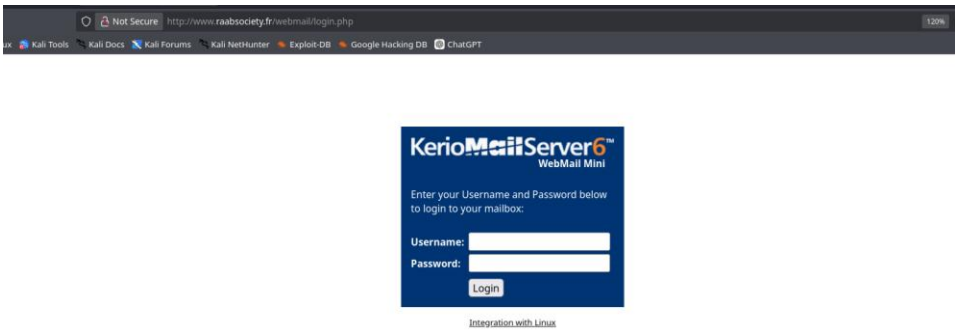
Ensuite, on tape l'url suivante **http://www.raabsociety.fr** sur le poste du client 1et on se connecte en tant qu'Alice (mot de passe : user1), on va ensuite envoyer un simple mail à bob@raabsociety.fr, en se connectant sur le poste de Bob (client 2), on verra qu'on a bien reçu le message qu'on vienne tout juste d'émettre depuis le poste d'Alice…



- **Q2 : URL utilisée par Alice :**

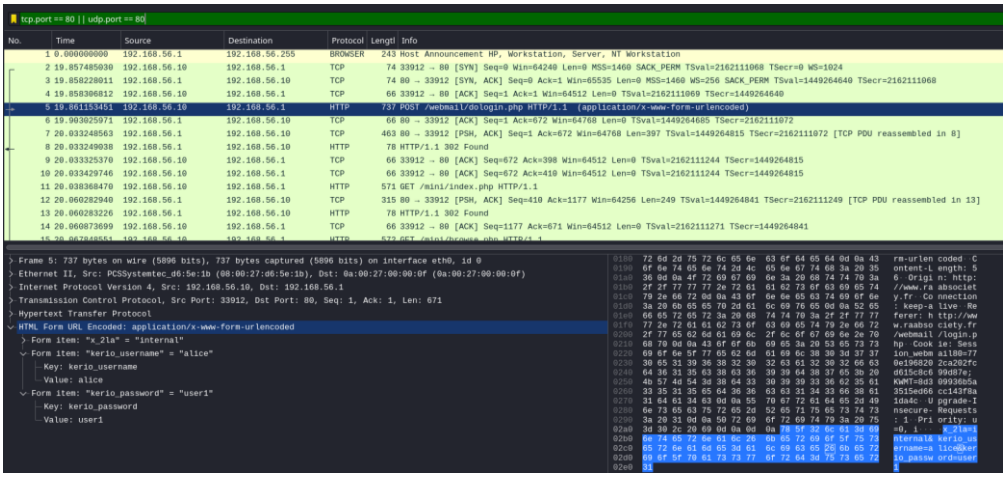
L'URL utilisée par Alice pour accéder au Webmail en HTTP (port 80) est **http://www.raabsociety.fr** ou **http://www.raabsociety.fr:80** (le port 80 est généralement implicite dans ce cas)

Et selon ma config, mon Kerio publie le WebMail sur un chemin spécifique, l'URL exacte observée devient alors **http://www.raabsociety.fr/webmail/login.php**



- **Q3 : Peut-on capturer login/mot de passe ? Est-ce en clair ?**

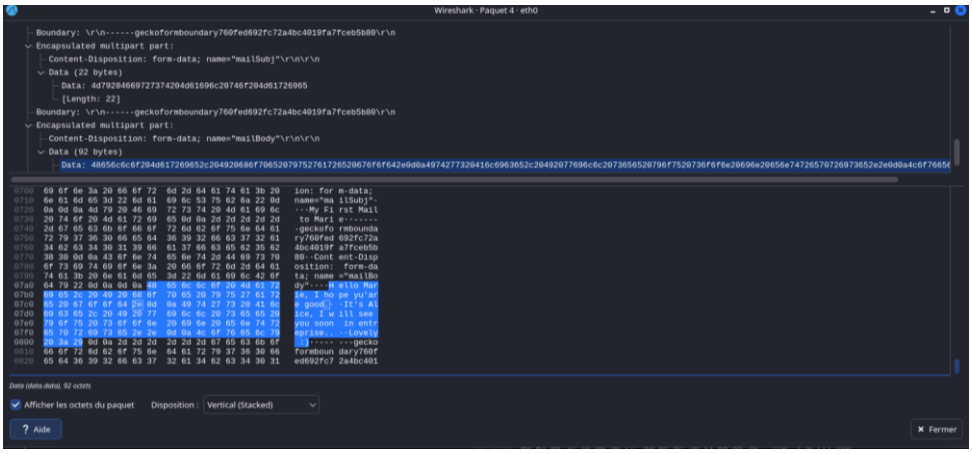
Lors d'un accès en HTTP (port 80), l'authentification transite sans chiffrement TLS, ce qui rend généralement possible la capture des informations d'authentification sur un réseau où l'on peut sniffer le trafic. Avec Wireshark (filtre tcp.port == 80 ou http), On doit vérifier si l'identifiant et/ou le secret d'authentification apparaît dans les requêtes HTTP (ex: paramètres de formulaire, en-têtes, etc.)



On peut voir les 2 champs kerio_username et kerio_password capturés sur Wireshark, ils sont en clair et toute personne lançant un sniffer sur le réseau pourra y avoir accès... Pas du tout sécurisé !

- **Q4 : Peut-on lire le contenu du message ?**

On nous demande de vérifier si le contenu du message envoyé via le Webmail est lisible lors de la capture réseau avec Wireshark... En HTTP, le contenu des échanges applicatifs (pages, actions, envois de message) n'est pas chiffré (donc pas sécurisé), donc il est possible de retrouver des éléments du message (sujet, corps, destinataire) dans la capture, selon la façon dont l'application web l'envoie.



A travers cette capture Wireshark, au niveau du champs nommé mailBody, on peut lire le message en clair dans la zone bleue (arrière-plan bleu), où l'on peut voir **Hello Marie, I hope yu'are good. It's Alice, I will see you soon in entreprise. Lovely :)** sur le côté droit, et sa traduction en octale sur le coté gauche.

Q5 : Solution pour sécuriser l'accès Webmail

Solution proposée : activer et imposer l'accès au Webmail en HTTPS (HTTP over TLS) au lieu de HTTP, afin de chiffrer l'authentification et les contenus échangés entre le navigateur et le serveur. Cela implique de configurer un certificat côté serveur (auto-signé pour un lab, ou CA interne/valide), puis de rediriger ou bloquer le port 80 au profit du port HTTPS (selon la politique retenue).

- **Q6 : Intérêt du Webmail vs client natif :**

Le Webmail permet l'accès aux emails depuis un simple navigateur (pas besoin d'installer/configurer un client lourd), ce qui facilite la mobilité et l'accès depuis des postes variés. En contrepartie, l'expérience dépend du navigateur et de la configuration web, et la sécurité repose fortement sur le bon paramétrage HTTP/HTTPS et des protections associées.

IV. Partie 3 - Accès aux emails via un client natif et le protocole POP3 :

- **Q1 & Q2 : Installation et configuration de Thunderbird sur mes VMs Clients :**

Sur chaque VM client (Kali Linux), on doit installer Thunderbird s'il n'est pas encore présent, pour ce faire, on remet une deuxième interface réseau à notre machine cliente, cette interface sera connectée en mode NAT, afin d'y avoir internet dans le but de pouvoir installer les packages de Thunderbird… pour l'installer, on ouvre un terminal et on tape les deux commandes suivantes dans cet ordre :

sudo apt-get update

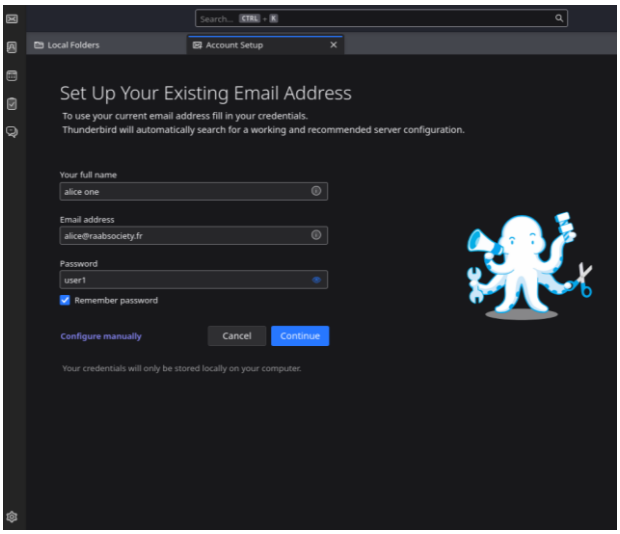
sudo apt install thunderbird

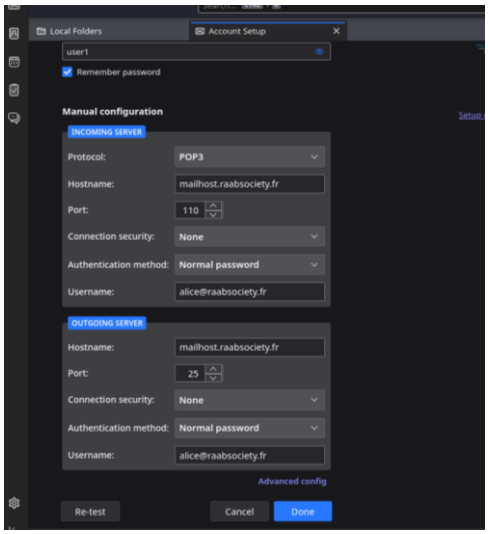
Ensuite, on édite le fichier /etc/hosts sur nos VMs clientes en utilisant la commande **sudo nano /etc/hosts**, en rajoutant la ligne suivante

192.168.56.1 mailhost.raabsociety.fr www.raabsociety.fr

Dans notre cas, 192.168.56.1 correspond à l'adresse IP du serveur WebMail, qui est elle-même celle de l'hôte Windows (sur l'interface connectée en Host-Only).

Ensuite, on lance Thunderbird (en tapant la commande thunderbird depuis un terminal client) et on configure le compte de **Alice** par exemple comme ci-dessous :





On voit bien que sur le serveur entrant, on utilise le protocole POP3 fonctionnant par défaut sur le port n°110, et sur le serveur sortant, on utilise plutôt le protocole SMTP qui fonctionne sur le port n°25. La logique de ces choix revient aux faites :

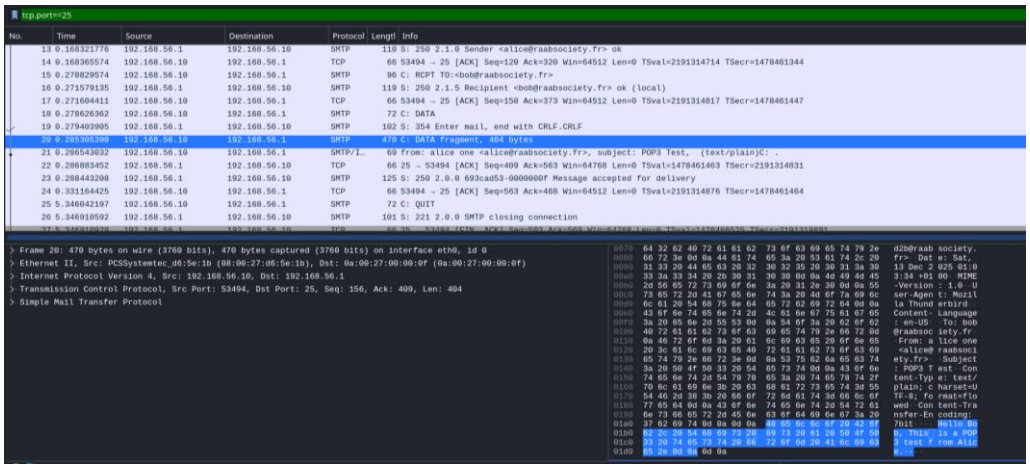
- SMTP (Simple Mail Transfer Protocol) est un protocole d'envoi de mails, c'est le protocole utilisé pour déposer un mail sur le serveur de messagerie (qui le livre ensuite à destination)
- POP3 (Post Office Protocol v3) est un protocole de réception de mails, c'est le protocole utilisé par le client pour télécharger les mails reçus stockés sur le serveur vers la boite locale.

- **<u>Q3 :</u> Envoi d'un email (Alice → Bob) et analyse SMTP :**

D'abord, après avoir lancé la capture (sniffing) sur Wireshark, on commence par rédiger et envoyer le message… Dans Thunderbird sur la VM Cliente de Alice, on compose un nouveau message à **bob@raabsociety.fr** ayant comme objet **POP3 test**, et pour corps **hello Bob, this is a POP3 test from Alice.** Et puis on clique sur le bouton **Envoyer ;**

Après l'envoi de ce message du coté de Alice, la capture Wireshark sur la VM d'Alice après application d'un filtre SMTP (tcp.port==25) montre clairement que le protocole SMTP a été utilisé pour envoyer le mail sur le server mail, comme on peut le voir sur la zone avec un arrière-plan bleu, où l'on peut voir le paquet SMTP contenant le contenu du message, comme on peut voir son contenu tout en bas…

Et donc pour répondre aux questions posées lors de l'énoncé de ce TP ;



3.1. Protocole applicatif utilisé par le client Alice pour envoyer son email ? **Le protocole applicatif s'agit du protocole SMTP (Simple Mail Transfer Protocol)**

3.2. Protocole de transport (TCP ou UDP) ? **Le protocole de transport utilisé est TCP port 25. SMTP utilise toujours TCP pour garantir la livraison**

3.3. Port client ? **Port éphémère (dynamique) attribué par le système client, généralement supérieur à 1024 (> 1024), par exemple, dans notre capture Wireshark, le paquet SMTP transportant le message (celui-ci-dessus) utilise le port source 53494 (comme on peut le voir sur l'illustration précédente ➔ Src Port : 53494.**

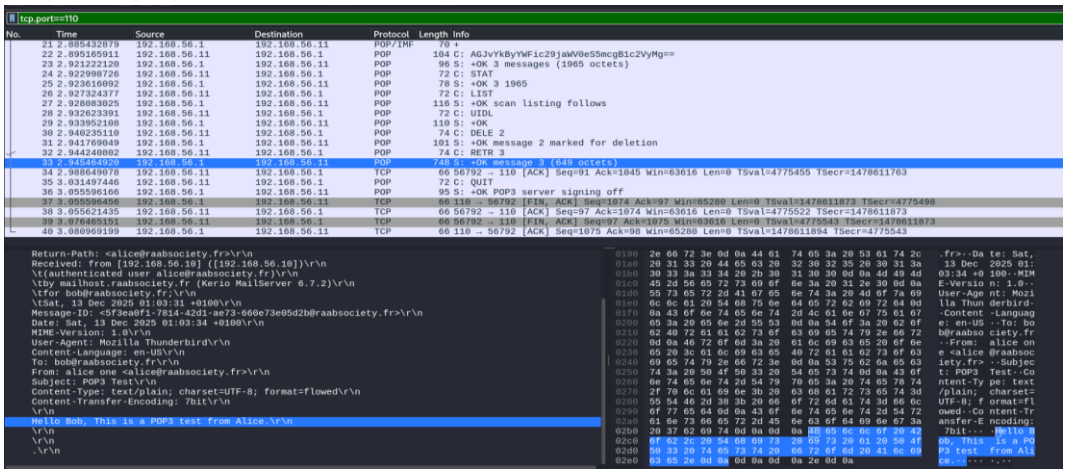
3.4. Port serveur ? **C'est le port 25 qui correspond au port SMTP par défaut qui est utilisé dans mon cas, d'après la capture Wireshark sur la machine d'Alice, on peut voir ➔ Dst Port : 25.**

3.5. Peut-on capturer login/password ? En clair ? **OUI. En SMTP non chiffré, l'authentification se fait souvent en AUTH PLAIN ou AUTH LOGIN, qui sont des encodages Base64 facilement décodables (et non pas du chiffrement), et ce comme on peut voir sur la capture Wireshark de la VM d'Alice, où l'on peut lire très clairement le contenu du paquet SMTP tout en lisant le contenu du mail, son objet, son destinataire, et toutes les informations concernant ce message…**

3.6. Peut-on lire le contenu du message ? **OUI. Le corps du message transite en clair dans les commandes SMTP (après DATA) comme on peut le voir sur la capture Wireshark dans la VM d'Alice sur la zone bleue où l'on reconnait clairement le contenu de notre message Hello Bob qui se trouve dans le paquet SMTP même…**

- **<u>Q4 :</u> Réception du message (Bob) et analyse POP3**

Du côté du récepteur du message, sur la VM de Bob, après avoir lancé une capture Wireshark on peut ordonner Thunderbird à récupérer les nouveaux emails depuis le server mail, on constate désormais que le protocole POP3 a été utilisé lors de cette procédure, pour mieux voir tout ça, on peut appliquer un filtre POP en mettant un filtrage Wireshark sur tcp.port==110, ce qui nous permettra d'avoir la capture ci-dessous :



4.1. Protocole de transport pour recevoir ? **C'est le protocole TCP (Transmission Control Protocol) qui est utilisé sous le port 110 réservé par défaut à POP.**

4.2. Port Client ? **Port éphémère (> 1024). Dans notre cas c'est le port 56792 qui est utilisé comme on peut voir → Dst Port : 56792 (port client = port destination, car POP est utilisé pour récupérer des emails depuis un MailServer du genre Kerio MailServer vers le client du genre VM Client Bob, donc la source de ce type de paquets est le MailServer, leurs destination est le client qui demande de récupérer ses emails).**

4.3. Port Serveur ? **C'est le Port 110 (port correspondant au port POP3 standard par défaut).**

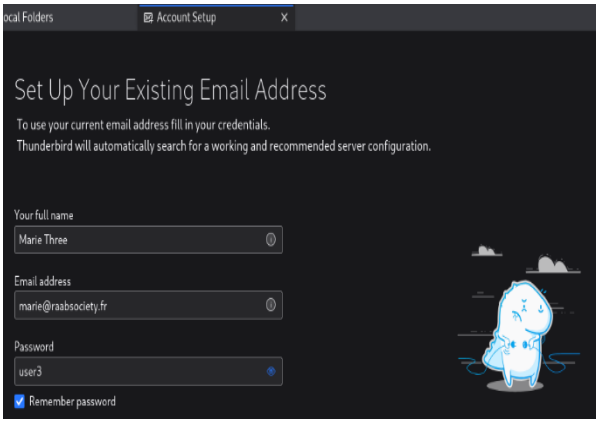
4.4. Peut-on capturer login/password ? En clair ? **Oui, POP3 non chiffré transmet les identifiants en clair avec les commandes USER et PASS, preuve : on peut lire le contenu du message récupéré depuis le MailServer où l'on distingue très clairement notre contenu (Hello Bob, This is a POP test from…) dans notre paquet POP, ce contenu est sous la zone bleue dans l'illustration, où l'on peut voir son encodage base 64 sur la gauche et son contenu clair à droite…**

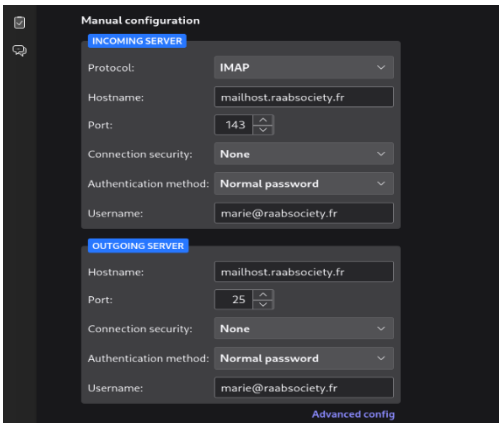
4.5. Peut-on lire le contenu du message ? **Oui, même chose. Le message est téléchargé en clair via la commande RETR. On peut le voir sur la capture Wireshark de la VM de Bob qui est juste ci-dessus…**

V. Partie 4 : IMAP :

Avant de commencer cette partie on doit commencer par une petite préparation, on doit installer le client mail nommé Thunderbird en suivant le même processus que lors de la partie précédente, après on se connecte au compte Marie (Identifiant : marie@raabsociety.fr, Mot de passe : user3) en utilisant IMAP (port 143) avec SMTP (port 25). Ensuite, on peut désormais lancer Wireshark (ou tcpdump) pour sniffer et observer les échanges avec le MailServer.

Afin de mieux comprendre cette préparation à faire, on accompagne cette petite explication de l'image ci-dessous où on peut voir l'ensemble de la configuration choisie pour cette partie du TP.



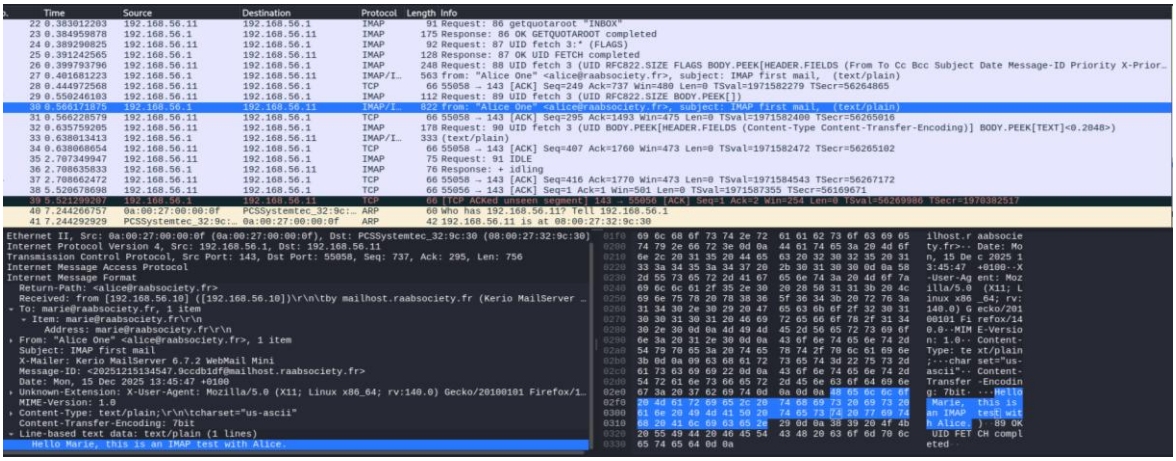


- **Q4.1. Réception d'un mail via IMAP…**

On envoie un mail depuis Alice vers Marie avec le texte demandé : Hello Marie, this is an IMAP test with Alice. Et puis dans la capture Wireshark préalablement lancée, on observe ce qui suit :

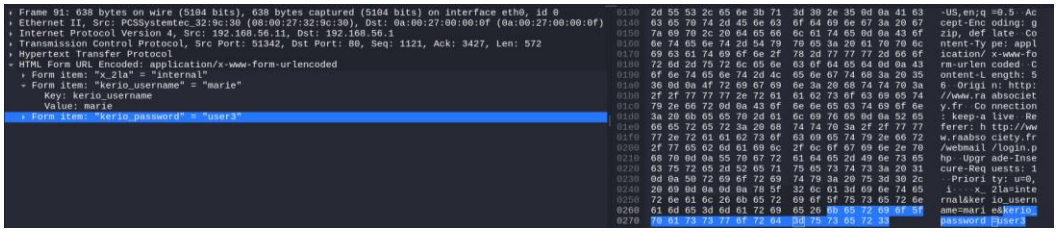
- Transport utilisé : TCP
- Port client (Dans cet exemple le client correspond à Marie) : C'est un port éphémère (>1024), dans mon cas c'est le port 55058 qui est utilisé…
- Port serveur : c'est le port 143 (par défaut le port occupé par IMAP)

On peut désormais voir toutes ces informations sur la capture ci-dessous :



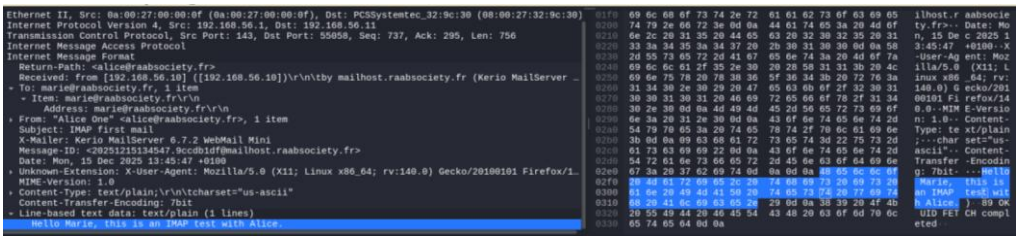
- **Q4.2. Identifiants interceptables ? Clair ou non ?**

En IMAP non chiffré (port 143, sans TLS), l'authentification apparaît en clair car aucun chiffrement ne sera appliqué sur les messages en transit sur le réseau, et c'est le protocole HTTP qui sera utilisé pour transmettre l'identifiant et le mot de passe en clair du client vers le MailServer pour qu'il analyse s'il existe un compte correspondant à cette paire, et ce, on peut le voir sur la capture Wireshark ci-dessous ;



- **Q2.3. Contenu du message lisible ?**

En IMAP non chiffré, le contenu et les en-têtes sont bien visibles (FETCH BODY, etc.). Dans Wireshark, on repère la réponse IMAP avec le corps du message où figure hello Marie, this is an IMAP test with Alice… comme on peut le voir dans la capture ci-dessous.



- **Q4.4. Tableau des différences POP3 vs IMAP4 ?**

	POP3	**IMAP**
Rôle côté client	Récupère les mails en les **téléchargeant** depuis le serveur.	Consulte/synchronise les mails en les gardant **sur le serveur**.
Port serveur	TCP **110**.	TCP **143.**
Effet pratique	Boîte surtout “locale” (risque de désynchronisation multi-postes si suppression après download).	Boîte “centralisée” (multi-appareils cohérents : mêmes dossiers/états).

- **Q4.5. nslookup : serveurs SMTP de l’école et DNS utilisé :**

```
C:\Users\raabr>nslookup -type=mx u-paris.fr
Serveur :   UnKnown
Address:  fe80::a81a:f1ff:fe04:a664

Réponse ne faisant pas autorité :
u-paris.fr      MX preference = 10, mail exchanger = mx04.cloud.vadesecure.com
u-paris.fr      MX preference = 10, mail exchanger = mx01.cloud.vadesecure.com
u-paris.fr      MX preference = 10, mail exchanger = mx02.cloud.vadesecure.com
u-paris.fr      MX preference = 10, mail exchanger = mx03.cloud.vadesecure.com

C:\Users\raabr>nslookup
Serveur par dÚfaut :   UnKnown
Address:  fe80::a81a:f1ff:fe04:a664

> server
Serveur :   UnKnown
Address:  fe80::a81a:f1ff:fe04:a664

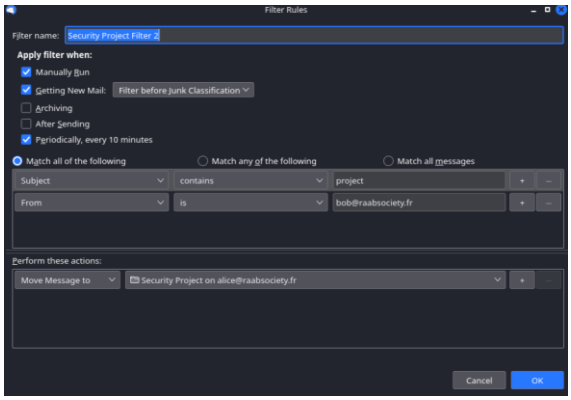
*** UnKnown ne parvient pas à trouver server : Non-existent domain
```

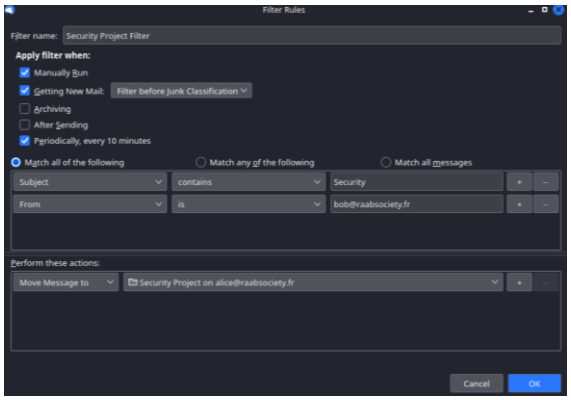
- **Q4.6. Dossier et filtre “Security Project” pour Alice :**

Pour commencer, on crée un dossier “Security Project” dans le client mail d’Alice, pour ce faire on fait clic doit au niveau de la barre latérale gauche sur Thunderbird, et puis new folder, on le nomme Security Project, ensuite on va dans le filtrage des messages pour ensuite créer ces deux règles de filtrage suivantes :

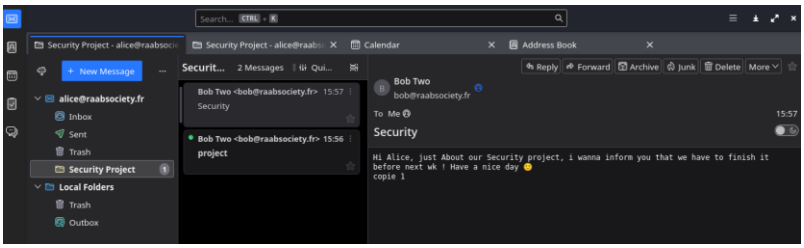
(on tombe sur les règles de filtrage en appuyant clic gauche sur alice@raabsociety.fr, autrement dit sur son propre mail au niveau de la barre latérale de Thunderbird)

- Règle 1 (à gauche) : Si le sujet contient project ET que l’expéditeur est Bob, alors classer dans le dossier “Security Project”.
- Règle 2 (à droite) : Si le sujet contient Security ET que l’expéditeur est Bob, alors classer dans le dossier “Security Project”.





Ensuite, on peut procéder aux tests, pour effectuer les tests on essaie d’envoyer quelques messages depuis Bob ou depuis Marie aussi, de tous les messages que j’ai envoyé en variant bien mes tests, seuls les deux messages suivants ont fini dans le dossier Security Project, le premier car il contient Security et l’autre contient Project en sujet, et ils sont tous les deux, deux mails venant de la part de Bob !

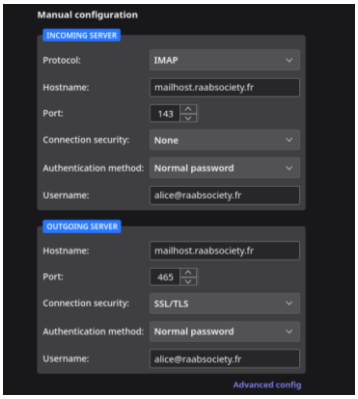


VI. Partie 5 : SMTP Over SSL (SMTPS):

On passe dans cette partie à sécuriser l’envoi des emails grâce avec SMTPS (en utilisant SMTP over SSL/TLS).

- **Q5.1. Configuration du client pour SMTPS ;** Dans le client mail d’Alice par exemple, le serveur sortant sera reconfiguré pour utiliser SMTP over SSL (SMTPS) afin d’envoyer un message à Bob avec l’objet et le contenu **test sending message with SMTPS** afin de tester l’efficacité d’utilisation du SMTPS.

Le serveur SMTP reste mailhost.raabsociety.fr comme configuré dans le fichier /etc/hosts, mais le port utilisé cette fois est celui du service SMTPS sécurisé (port 465 par défaut) avec une touche TLS activée coté client. Cette capture ci-dessous montre la config exacte à appliquer pour avoir le SMTPS.



- **Q5.2. Port du serveur SMTPS ?** Le port standard utilisé par SMTPS (SMTP directement sur SSL/TLS) est le port 465/TCP, ce qui correspond exactement au port utilisé par défaut par SMTPS. Dans la configuration du client, le port du serveur SMTPS a donc été positionné sur 465 afin d'établir une connexion chiffrée de bout en bout entre le client d'Alice et le MailServer (mailhost.raabsociety.fr). Et ça on peut le vérifier sur la capture Wireshark ci-dessous où l'on peut voir dst.port : 465 !!

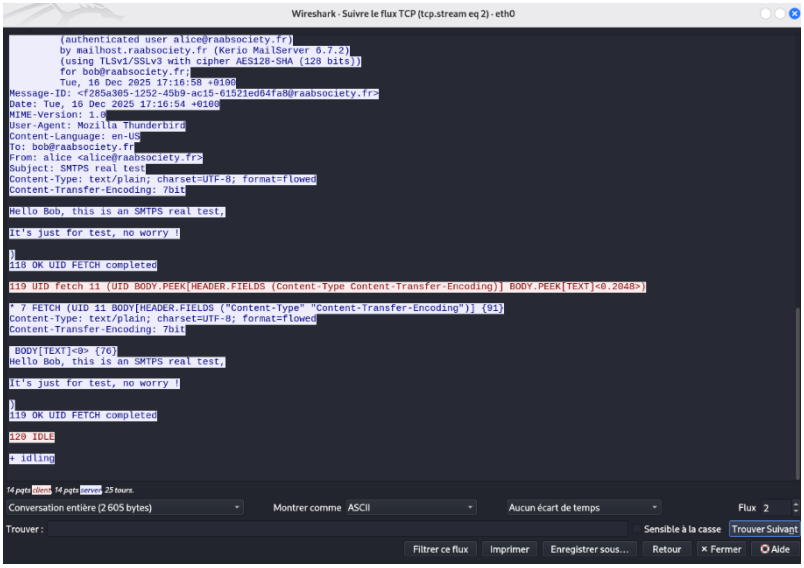


- **Q5.3. Login/mot de passe et contenu en SMTPS (CR)**

- **Observation 1 (côté envoi, Alice → serveur, SMTPS) :** en configurant l'envoi d'Alice en SMTPS, les échanges SMTP sont encapsulés dans TLS ; dans Wireshark, on voit un flux TLS et on ne peut plus lire ni les identifiants SMTP (AUTH) ni le contenu du message.



- **Observation 2 (côté réception, serveur → client récepteur en IMAP classique) :** puisque Bob relève le message via IMAP non chiffré (port 143), alors sur ce lien serveur → client on peut sniffer et voir en clair le contenu du mail qui lui a été destiné depuis Alice même si elle de son coté, elle a bien sécurisé la communication.



Donc, on peut désormais dire que SMTPS protège bien l'envoi, mais ne protège pas la consultation/récupération côté récepteur tant que IMAP/POP n'est pas aussi chiffré (en IMAPS/POP3S).

- **Q5.4. Types de chiffrement / clés utilisés par SMTPS :**

Une connexion SMTPS utilise le protocole TLS qui met en place :

- Un échange de clés asymétriques (clé publique/privée du serveur, éventuellement certificat autosigné X.509) pour négocier un secret de session.
- Puis un chiffrement symétrique (tel AES) avec cette clé de session pour ensuite chiffrer tout le trafic SMTP.

- **Q5.5. Services de sécurité fournis par une connexion SMTPS :**

Une connexion SMTPS fournit les services de sécurité suivants :

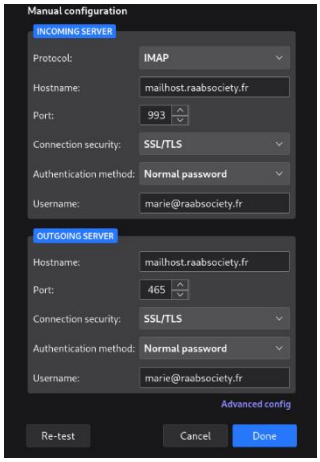
- **Confidentialité :** le contenu des messages et les identifiants ne sont plus lisibles par un attaquant qui sniffe le réseau.
- **Intégrité :** TLS détecte les modifications non autorisées des données en transit.
- **Authentification du serveur :** grâce au certificat (auto-signé dans mon cas) présenté par le serveur, le client peut vérifier l'identité du serveur (ou dans un contexte réel avec CA).

VII. Partie 6 : Email Communications Security with IMAPS (IMAP over SSL) :

- **Q6.1. Configurer Marie pour relever via IMAPS :** Le client mail (Thunderbird) de Marie a été configuré pour récupérer ses emails via IMAPS (IMAP over SSL) depuis le serveur mailhost.raabsociety.fr (Kerio), avec les identifiants marie@raabsociety.fr avec comme mot de passe : user3.

  La connexion utilise IMAP sur SSL/TLS (IMAPS) afin de chiffrer l'authentification et les données échangées entre le client et le serveur.

  Remarque de mise en œuvre : le serveur Kerio utilisé (6.7.2) ne propose que TLSv1.0 et un certificat auto-signé ; pour permettre la connexion depuis Thunderbird, le minimum TLS côté client a dû être abaissé et le certificat accepté manuellement.

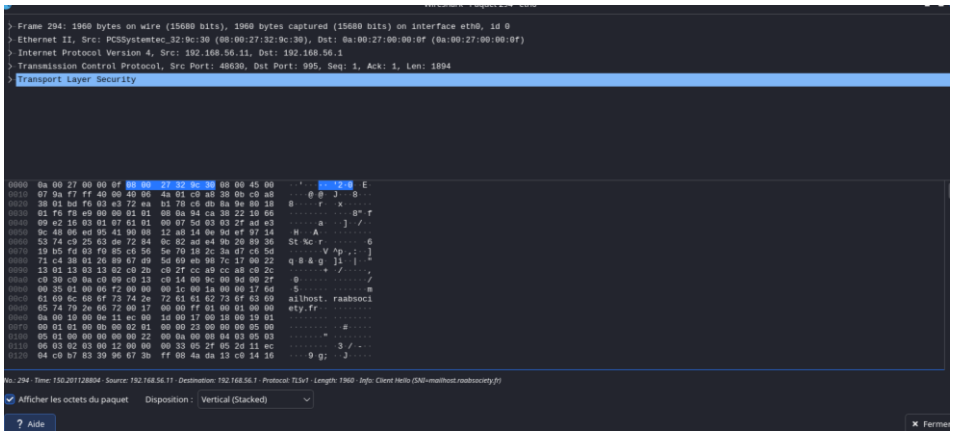


- **Q6.2. Port du serveur IMAPS :** Le port serveur utilisé en IMAPS (IMAP over SSL/TLS) est 993/TCP.
- **Q6.3. Sniff : login/mdp et contenu en clair ?** Avec IMAPS, l'authentification (login/mot de passe) et le contenu des emails circulent dans un tunnel TLS, donc ils ne sont plus visibles en clair dans Wireshark (on observe du trafic TLS au lieu de commandes IMAP lisibles).

  La capture réseau montre une connexion TCP vers le port 993 et un handshake TLS ; les commandes IMAP de type LOGIN et les données FETCH/BODY ne sont pas lisibles en clair.



- **Q6.4. Port serveur en POP3S :** Le port serveur utilisé en POP3S (POP3 over SSL/TLS) est 995.



VIII. Partie 7 : END-TO-END Email Security with PGP :

Q7.1. Installation des prérequis : Dans le cadre de la partie 7, les outils PGP ont été mis en place sur le client en installant GnuPG (moteur OpenPGP), et ce en utilisant les deux commandes suivantes (sous mon client kali dans mon cas) :

sudo apt update

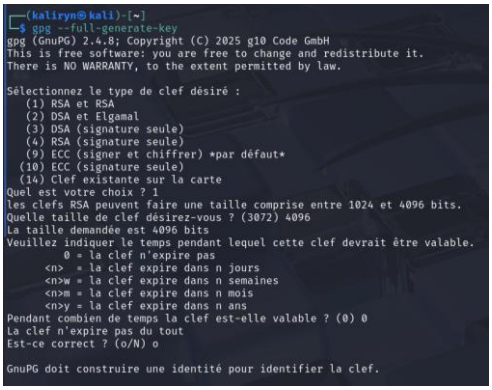
sudo apt install -y gnupg dirmngr

Le TP demande Enigmail, mais avec ma version actuelle de Thunderbird 140 ESR, l'extension Enigmail n'est plus utilisable comme avant car elle a été remplacée par le support OpenPGP intégré à Thunderbird (menu *End-To-End Encryption*), ce qui permet de réaliser les mêmes opérations (génération de clés, signature, chiffrement).

Ainsi, l'environnement client dispose bien de GnuPG et d'un module OpenPGP opérationnel dans Thunderbird pour la suite (signature et chiffrement PGP).

Q7.2. Création d'une paire de clés et publication de la clé publique :

- Pour créer une paire de clés avec GnuPG on suit la procédure ci-dessous :





- Ensuite, lors de la publication de notre clé publique d'Alice sur le serveur demandé, on s'en rend compte que le serveur pool.sks-keyservers.net est mort / indisponible, c'est pourquoi on décide d'utiliser un autre serveur de clés qui est keys.openpgp.org, et on procède justement de la façon suivante :



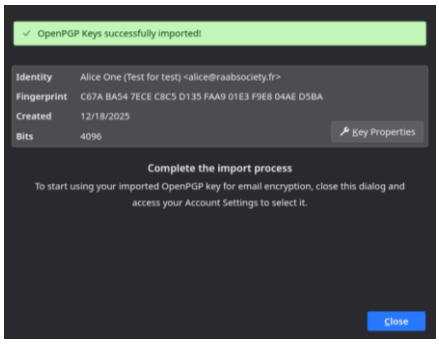
Q7.3. Configurer et envoyer un émail signé (Alice)…

- D'abord, on doit commencer par récupérer la privée d'Alice stockée localement dans le Keyring de GnuPG, pour l'importer sur son compte sur Thunderbird pour qu'on puisse signer ses e-mails avec… Pour ce faire, on doit taper la commande suivante :

gpg –-armor –export-secret-keys alice@raabsociety.fr > alice_privatekey.asc

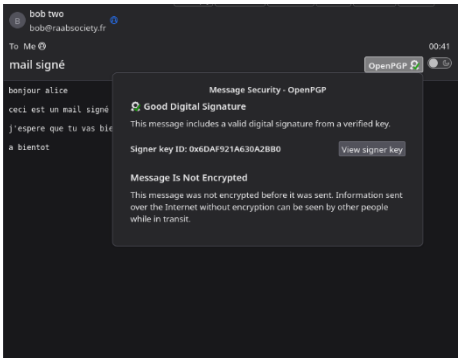
Cela exporte la clé privée d'Alice depuis le Keyring vers le dossier courant où a été tapée cette commande, ensuite plus qu'à l'importer sur Thunderbird sur le compte d'Alice ;

Afin d'importer la clé sur Thunderbird, on va sur le →Compte d'Alice,
Puis →Account Settings,
Et puis →End-To-End Encryption,
Et puis dans OpenPGP, on tape sur →Add Key,
Et puis →Import an existing OpenPGP key,
Puis on importe la clé exportée juste avant qu'on ait appelé alice_privatekey.asc,
Si tout est bien suivi, on verra l'écran suivant s'afficher :



Une fois cet écran affiché, la clé privée générée est bien associée à ce compte d'Alice…

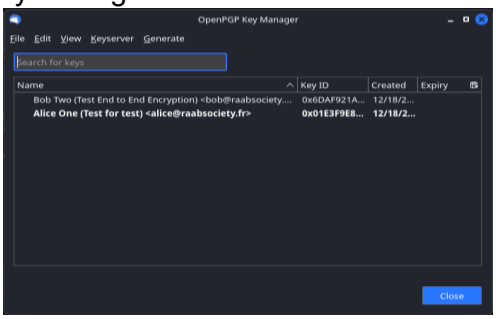
Ensuite, on choisit d'utiliser cette clé importée comme clé par défaut pour le compte d'Alice. Et puis à la rédaction de notre mail, on choisit l'option **Digitally sign** présente dans la section OpenPGP, et puis, plus qu'à envoyer notre mail !



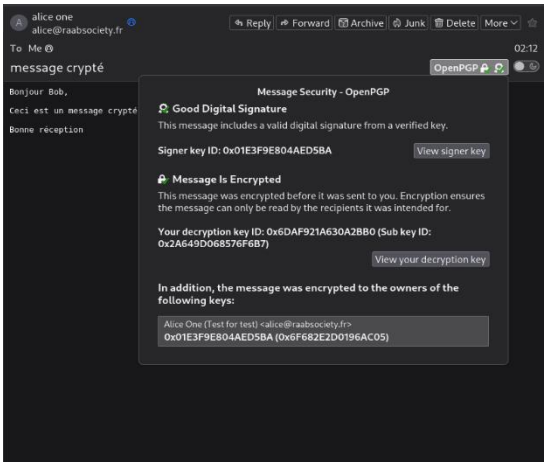
Q7.4. Envoyer un email chiffré avec PGP (Alice → Bob)..

D'abord, Pour chiffrer vers Bob, Alice doit avoir la clé publique de Bob dans son trousseau (Thunderbird ou GnuPG). Sans clé publique de Bob, Thunderbird ne peut pas chiffrer (il doit connaître à qui verrouiller le message).

- Etape 1 : Récupérer et importer la clé publique de Bob, depuis la machine de Bob, exporte sa clé publique puis importe-la chez Alice sur la machine de Alice.
  On doit avoir ça dans le key manager de Alice dans le End-To-End Encryption Setups



- Etape 2 : Chiffrer et envoyer l'email depuis Thunderbird (compte Alice) : D'abord on compose un nouveau mail vers bob@raabsociety.fr. Ensuite, dans la fenêtre de rédaction, on active Encrypt (OpenPGP) (chiffrement), on active aussi sign pour avoir chiffrement + signature, et puis on envoie le message...
  Dans le client mail de Bob (Thunderbird), On doit recevoir un message comme celui-ci (chiffré et signé) :



Q7.5. When and how is the public key retrieved by the receiver?

Le destinataire récupère la clé publique de l'expéditeur soit via un serveur de clés (recherche par adresse mail, KeyID ou fingerprint), soit via un échange direct (fichier .asc joint à un email, dépôt sur un site, etc).

Dans le TP, la logique attendue est : publication de la clé sur un keyserver qui est celui de keys.openpgp.org dans notre cas, puis import/recherche par le destinataire soit depuis Thunderbird directement, ou manuellement via un terminal en utilisant du GPG natif pour importer la clé au niveau local (keyring), et ce avant de pouvoir vérifier une signature ou chiffrer un mail.

7.6. What key is used to ensure the authenticity of emails?

L'authenticité (et l'intégrité via la signature) est assurée par la clé privée de l'expéditeur, utilisée pour signer le message.

Le destinataire vérifie ensuite cette signature avec la clé publique correspondante de l'expéditeur.

7.7. What key is used to ensure email confidentiality?

La confidentialité est assurée en chiffrant le message pour le destinataire à l'aide de la clé publique du destinataire (chiffrement PGP).

Seul le destinataire peut déchiffrer avec sa clé privée associée.

7.8. What is the hash function used by GnuPG?

GnuPG utilise une fonction de hachage dans le cadre des signatures (le hash est ensuite signé), et l'algorithme exact dépend de la configuration/clé (souvent SHA-256 sur des configurations modernes, parfois SHA-1 sur des environnements plus anciens).

7.9. Wireshark: are login/password protected? Explain why.

PGP ne protège pas directement les identifiants SMTP/IMAP (login/mot de passe), car il s'applique au contenu du message (payload) et pas à l'authentification des protocoles de transport.

Les identifiants sont protégés uniquement si la couche de transport est chiffrée (SMTPS/IMAPS).

En revanche, même si le transport n'est pas chiffré, le contenu d'un email chiffré PGP reste illisible dans Wireshark (on capture que du texte chiffré OpenPGP).

IX. Conclusion:

Les tests en HTTP/POP3/IMAP/SMTP non chiffrés ont montré qu'un attaquant en écoute peut intercepter des informations sensibles (authentification, contenu des messages), ce qui justifie la nécessité du chiffrement des protocoles de transport.

La mise en place de SMTPS et IMAPS/POP3S a permis de chiffrer les échanges client–serveur et d'empêcher la lecture en clair dans Wireshark, tout en soulignant qu'il faut sécuriser chaque lien (envoi et réception) pour éviter qu'un segment reste exposé.

Enfin, l'utilisation de PGP (signature et chiffrement) apporte des garanties complémentaires (authenticité, intégrité, confidentialité) directement sur le message, indépendamment du serveur de messagerie et du transport, ce qui constitue une approche robuste pour protéger les communications sensibles.